N

What is NestJS?

The only backend guide you'll ever need.

HM
Hafiz Mannan Siddiqui
Software Engineer | AI & ML

THE PROBLEM

Your Express code is a mess. Here's why.

Express.js

- ✗ No enforced structure
- ✗ Every dev does it differently
- ✗ Routes + logic + DB all mixed
- ✗ Becomes spaghetti code

NestJS

- ✓ Clean architecture enforced
- ✓ Same pattern for everyone
- ✓ Separation of concerns
- ✓ Scales 1 to 100 developers

Think of it this way:

Express = LEGO pieces. Build from scratch.

NestJS = IKEA furniture. Organized with instructions.

CORE CONCEPT

NestJS has only 3 building blocks.

Master these and you master NestJS. Seriously.

1 **MODULE** — The container. Groups related code.

↓ Module registers...

2 **CONTROLLER** — The receptionist. Handles HTTP requests.

↓ Controller delegates to...

3 **SERVICE** — The brain. Business logic + database.

Every feature = Module + Controller + Service.

Users? Same pattern. Payments? Same pattern. Always.

BUILDING BLOCK 1

Module — The Container

```
@Module({
  controllers: [UsersController],
  providers:   [UsersService],
})
export class UsersModule {}
```

Think of it as a labeled box:

📦 **"Everything about Users goes in here"**

- One module per feature
- Registers controllers + services
- All modules imported in app.module.ts
- Does nothing itself — just organizes

BUILDING BLOCK 2

Controller — The Receptionist

```
@Controller('users')
export class UsersController {

  @Get()          // GET /users
  findAll() {}

  @Post()         // POST /users
  create() {}

  @Put(':id')     // PUT /users/123
  update() {}

  @Delete(':id')  // DELETE /users/123
  remove() {}
}
```

> It **receives** requests. It does NOT contain logic. It calls the service.

BUILDING BLOCK 3

Service — The Brain

```
@Injectable()
export class UsersService {

  constructor(private db: PrismaService) {}

  findAll() {
    return this.db.user.findMany();
  }

  create(data) {
    return this.db.user.create({ data });
  }
}
```

This is where the **REAL work** happens:
→ Database queries → Business logic → Calculations

HOW IT WORKS

Every request follows 5 steps.

No exceptions. This is the entire architecture.

1. **Browser:** GET /users/469
2. **Controller:** @Get(':id') → findOne(469)
3. **Service:** db.user.findUnique({ id: 469 })
4. **Database:** SELECT * FROM users WHERE id = 469
5. **Response:** { id: 469, name: "Abdul Mannan" }

Browser → Controller → Service → Database → Response

Once you see this, you can read any NestJS codebase.

SAVE THIS

Decorator Cheat Sheet

Bookmark this slide. You'll use it every day.

`@Controller('path')`
Define the base URL for routes

`@Get() @Post() @Put()`
Map to HTTP methods

`@Param('id')`
Extract from URL: /users/:id

`@Query('page')`
Extract query: ?page=2

`@Body()`
Extract JSON request body

`@Request()`
Full request (for user auth)

`@UseGuards(Guard)`
Add security checkpoint

`@Injectable()`
Allow dependency injection

COMPARISON

NestJS vs Express

When to use what? Here's my honest take.

Use Express when:

→ Small project / quick prototype
→ You want full control
→ Solo developer
→ Learning Node.js basics

Use NestJS when:

✓ Team project (2+ developers)
✓ Enterprise / production app
✓ Need auth, validation built-in
✓ TypeScript-first workflow
✓ Long-term maintenance matters

My recommendation: Learn Express first to understand the basics. Then switch to NestJS for anything serious.

Found this useful?

→ NestJS, React, TypeScript deep dives

→ Real-world project breakdowns

→ AI & Machine Learning tutorials

→ System design for beginners

Hafiz Mannan Siddiqui

linkedin.com/in/hafiz-mannan-siddiqui

💾 Save ♻️ Repost 💬 Comment